Unit 3: Knowledge Representation

Artificial Intelligence - Exam Revision Notes
Based on the Unit 3 slides (88 slides)

1. Basics of Knowledge Representation

Knowledge Representation (KR) means encoding information about the world into a format an AI system can understand and use to reason and make decisions. It bridges the gap between raw data and meaningful reasoning.

Elements of KR

- **Objects and entities** - physical/abstract things and their properties (cars, people, traffic signs).
- **Events and actions** - what happens and what the AI can do (game moves).
- **Relationships and hierarchies** - "is part of", "is related to" (dog is a type of animal).
- **Facts and rules** - facts are specific information; rules define the logic (IF fever AND sore throat THEN may be flu).
- **Uncertainty** - incomplete or noisy real-world data (stock prediction, diagnosis).

Types of Knowledge

Type	Meaning	Example
Declarative	"Knowing what" - static facts	Paris is the capital of France
Procedural	"Knowing how" - dynamic, action-oriented	Solving an equation, baking a cake
Structural	Relationships between concepts (hierarchies/networks)	Poodle → dog → animal
Meta	Knowledge about knowledge - which knowledge to use	Which symptoms to prioritise in diagnosis
Heuristic	Experience-based rules of thumb, not guaranteed	Website slow → refresh the page

Cycle of KR in AI

Perception → Learning → KR & Reasoning → Planning → Execution (feedback from execution goes back into the cycle).

Stage	Description	Example
Perception	Gather data from sensors, cameras, inputs, databases	Self-driving car detects pedestrians
Learning	Find patterns (supervised/unsupervised/reinforcement)	Learn from sales data to predict trends
KR & Reasoning	Organise learned data into structured knowledge and reason	Medical system links symptoms to diseases
Planning	Choose best actions given goals and constraints	Robot finds path avoiding obstacles
Execution	Carry out the plan, receive feedback	Robot picks and moves objects

Applications of KR

Expert systems, Natural Language Processing (chatbots, translation), Robotics, Semantic Web (ontologies), Cognitive computing (IBM Watson).

2. Approaches to Knowledge Representation

Approach	Description	Strengths	Weaknesses
Simple relational	Facts in tables/columns, like a database	Simple, easy to implement	Little inference; can't handle complex relations
Inheritable	Hierarchies; entities inherit properties (dog → mammal)	Efficient, reduces redundancy	Hard to represent exceptions
Procedural	Sequences of steps to do a task (robot: locate, move, grip)	Good for automation	Poor for complex reasoning
Inferential	Derive new info from facts + rules (All humans mortal; Socrates human → mortal)	Enables logical deduction	Computationally expensive; weak with uncertainty

3. Requirements of a Good KR System

Remember the 5: Representational Adequacy, Inferential Adequacy, Inferential Efficiency, Acquisitional Efficiency, Consistency.

Requirement	Meaning	Challenge
1. Representational adequacy	Represent all relevant knowledge (facts, relationships, rules) of the domain	Handle vast knowledge without becoming too complex
2. Inferential adequacy	Generate new knowledge via deduction, induction, abduction	Efficient algorithms for large data
3. Inferential efficiency	Make inferences quickly using memory/processing optimally	Balance complexity vs speed (e.g. real-time stock trends)
4. Acquisitional efficiency	Easily acquire and integrate new knowledge (e.g. new FAQs)	Stay scalable and adaptive
5. Consistency	No contradictions among facts, rules, relationships	Detect and resolve inconsistencies as the KB grows

4. Knowledge Representation Techniques

A. Logical Representation

A language with definite rules dealing with propositions, no ambiguity, with precisely defined **syntax and semantics** supporting sound inference.

Advantages	Disadvantages
Clear, precise; removes ambiguity Allows inference and reasoning Mathematically accurate; used in expert systems, theorem proving, NLP	Slow and complex (esp. First-Order Logic) Doesn't handle uncertainty well Rigid - needs exact definitions Harder to control as KB grows

B. Propositional Logic (PL)

Deals with statements (**propositions**) that are either **True (1)** or **False (0)**; also called **Boolean logic**. Complex statements are built from atomic propositions (e.g. P: "The light is on", Q: "The window is open").

Building blocks

- **Propositions** - statements classifiable as true/false ("The sky is blue" = True).
- **Logical connectives** - combine propositions (below).
- **Truth tables** - list all combinations of truth values to find the value of a complex proposition.
- **Compound propositions** - e.g. "If raining and heater on, room is warm" = $(P \wedge Q) \rightarrow R$.

Connective	Symbol	True when...
AND	∧	both propositions are true
OR	∨	at least one proposition is true
NOT	¬	(reverses) the proposition is false
IMPLIES / IF-THEN	→	true unless the first is true and the second is false (rain → wet ground)
IF AND ONLY IF	↔	both have the same truth value (both true or both false)

Inference rules

- **Modus Ponens:** if P → Q and P are true, then Q is true.
- **Modus Tollens:** if P → Q and ¬Q are true, then ¬P is true.

Laws / Properties

Law	Rule
De Morgan's 1st	¬(P ∧ Q) ≡ (¬P ∨ ¬Q)
De Morgan's 2nd	¬(P ∨ Q) ≡ (¬P ∧ ¬Q)
Commutative	P ∧ Q ≡ Q ∧ P; P ∨ Q ≡ Q ∨ P
Associative	(P ∧ Q) ∧ R ≡ P ∧ (Q ∧ R); (P ∨ Q) ∨ R ≡ P ∨ (Q ∨ R)
Distributive	P ∧ (Q ∨ R) ≡ (P ∧ Q) ∨ (P ∧ R); P ∨ (Q ∧ R) ≡ (P ∨ Q) ∧ (P ∨ R)

Applications of PL	Limitations of PL
Knowledge representation (expert systems) Automated reasoning (Modus Ponens/Tollens) Problem solving and planning (STRIPS) Decision making NLP (semantic parsing)	Can't represent relationships between objects No handling of uncertainty (only true/false) Limited expressiveness (no time/dynamic events) Scalability issues as propositions grow

C. Predicate Logic / First-Order Logic (FOL)

An **extension of propositional logic** that expresses relationships between objects and their properties. It adds **predicates, variables, constants and quantifiers**. Example: "John is the father of Mary" = Father(John, Mary).

- **Predicate** - function returning true/false based on its arguments, e.g. IsHungry(John).
- **Variable** - placeholder for objects, e.g. x in IsHungry(x).
- **Constant** - a specific object, e.g. John.
- **Predicate symbol** - the property/relationship (IsHungry, Married).
- **Arity** - number of arguments: IsHungry(x) = 1, Married(x, y) = 2, X(a, b, c) = 3.

Quantifiers

Quantifier	Meaning	Example	Negation
Existential ∃	At least one object satisfies the condition	∃x IsHungry(x) - someone is hungry	¬∃x IsHungry(x) - no one is hungry
Universal ∀	Condition holds for all objects	∀x (IsHuman(x) → IsMortal(x)) - all humans are mortal	¬∀x ... - at least one exception

Examples

- Simple: IsHungry(John). Equality: E(x, y) ≡ (x = y). Mathematical: X(a, b, c) ≡ (a + b + c = 0).
- Relationship: M(x, y) ≡ x is married to y.
- Multiple quantifiers: ∀x ∃y Parent(x, y) - every person x is a parent of some y.

Applications: NLP (chatbot intent), robot planning (∃x TaskRequires(x, Charging)), expert systems (if-then rules for diagnosis).

PL vs FOL: PL handles only true/false statements. FOL adds objects, relations and quantifiers (∀, ∃), so it is more expressive.

D. Semantic Networks

A **graph-based** technique: concepts are **nodes**, connected by relationships shown as **edges/arcs**. It mirrors how humans store and link information through associations.

Components

- **Nodes** - concepts/entities (Car, Apple, Happiness).
- **Edges (links)** - relationships (Bird → Animal).
- **Labels** - nature of the relation: is-a, has-part, can-do, owned-by.
- **Properties/attributes** - Color, Size, Function.
- **Hierarchical structure** - parent-child arrangement for inheritance.

Types

Type	Description	Example / Use
Hierarchical	Strict parent-child tree	Animal → Mammal/Bird; taxonomies
Heterarchical	Flexible web-like, no strict levels	Maths ↔ Physics/Statistics/CS; knowledge graphs
Propositional	Nodes are propositions/statements	"Sky is blue" + evidence; expert systems
Multilevel	Layers of abstraction; node expands into sub-network	Product → components, suppliers, costs
Associative / Conceptual	Mental associations between concepts	Summer → sunshine, beach, vacation
Dynamic	Adapts/evolves with new information	Recommendation systems

Uses in AI

- **Knowledge representation** - structured, with context (Penguin is a Bird but doesn't fly).
- **Inference** - traverse links (Whale is a Mammal; Mammals are warm-blooded → Whales are warm-blooded).
- **Semantic search and retrieval** - context-based results, not just keywords.
- **Natural Language Understanding** - chatbots, virtual assistants, translation.

Knowledge sources: **extraction from text data** (NLP identifies entities/relations) and **integration from structured databases** (map schema to nodes/relations).

8 Steps to Build a Semantic Network (example: "Fruit Types")

Step	Action
1	Define the domain and purpose
2	Identify entities (nodes) - apple, banana, orange, grape
3	Determine relationships (edges) - is-a, color, taste, grows-in-region
4	Establish attributes (apple colour: red/green; taste: sweet/tart)
5	Create the network structure
6	Populate the network (manual entry, text extraction, databases)
7	Visualise and navigate (graph visualisation tools)
8	Maintain and update

Applications: NLP (Google Knowledge Graph), expert systems, machine translation (word sense), recommendation engines, information retrieval, ontology building. **Examples:** animal classification (is-a), food chain ("eaten by"), programming concepts (is-a / has).

E. Production Rules / Production Systems

A **rule-based model** that solves problems by applying predefined **IF-THEN** rules to the current state. Example: *If temperature > 30°C, then turn on the air conditioning.*

Components

- **Global database** - the central data structure (current state).
- **Set of production rules** - each has a precondition; if satisfied, the rule can be applied, and applying it changes the database.
- **Control system** - chooses which applicable rule to apply, resolves conflicts, stops when the termination condition is met.

Features

- **Simplicity** - uniform IF-THEN structure, readable.
- **Modularity** - knowledge in discrete pieces; add/delete without side effects.
- **Modifiability** - rules can be written in skeletal form and refined.
- **Knowledge-intensive** - knowledge base holds pure knowledge, no control info.

Classes of Production Systems

Class	Key idea
Monotonic	Applying a rule never prevents a later rule that was applicable earlier
Partially commutative	Any allowed permutation of a rule sequence gives the same final state (theorem proving)
Non-monotonic	For ignorable problems; no backtracking needed, so more efficient
Commutative	Changes are reversible and order isn't critical. Non-commutative systems (e.g. chemical analysis) have irreversible changes, so correct decisions are needed first time

Types of Production Systems

Type	Description	Examples
Rule-based	Apply if-then rules to data to deduce new info	Medical diagnosis, fraud detection
Procedural	Knowledge of how to perform tasks (steps)	Manufacturing control, IVR systems
Declarative	Facts about what something is; queried to decide	Siri/Alexa knowledge bases, configuration systems

Architecture (real-world problem solving)

- **Rule base (knowledge base)** - if-then rules; its richness decides effectiveness.
- **Working memory** - facts about the current problem state; dynamic, acts as short-term memory.
- **Inference engine** - the "brain"; uses pattern matching to apply rules to facts. Two modes: **Forward chaining** (facts → conclusion) and **Backward chaining** (goal → check if conditions can be met).
- **Conflict resolution** - decides which rule fires when many apply: **Specificity** (most specific rule), **Recency** (matches latest facts), **Priority levels** (highest weight).

Other points: modular (independent rule modules), inference-driven problem solving, **deterministic** (same output every time) vs **non-deterministic** (multiple rules/probabilistic outcomes). **Applications:** healthcare expert systems, finance (risk, credit, fraud), industrial automation, robotics/intelligent agents.

F. Frames

A **structured knowledge representation** technique that models a concept/object with its attributes, properties and relationships. A frame = **slots (attributes)** + **fillers (values)**.

- **Structured modelling** - organises knowledge into coherent units.
- **Attribute-value pairs** - slot = attribute, filler = value.
- **Hierarchical organisation** - frames can have subframes (inheritance).

- **Relationships** - captures how entities are connected.

Example: Person frame

Slots: Name, Age, Gender, Address, Phone → Fillers: John Smith, 35, Male, 123 Main Street, (555) 123-4567.

Example: Book frame (complete)

- **Slots:** Title "To Kill a Mockingbird", Author Harper Lee, Year 1960, ISBN, Genre Fiction.
- **Facets** (constraints): Publication Year - integer, range 1450 to current year; ISBN - 13-digit number.
- **Default values:** Genre = "Unknown" if not specified.
- **Procedures:** CheckAvailability, UpdateRecord.

Frames vs Ontologies

	Preferred when	Example
Frames	Modelling detailed attributes/values of objects; rich structured data in a specific domain	Medical diagnostic systems
Ontologies	Semantic clarity and interoperability between systems; standard terminology	Semantic web

Applications of Frames

Area	How frames are used
Expert systems	Frame per condition/problem; slots for symptoms/issues, fillers for data
NLP	Represent semantic structure (restaurant frame: cuisine, location, rating)
Robotics	Model objects and properties (cup: shape, material, size)
Diagnosis / troubleshooting	Frame hierarchy of equipment, components, symptoms, solutions
Financial analysis	Stock frame: Company, Price, Market Performance; match goals to investments

Frame problem, Frames and Scripts

- **Frame problem:** after an action, the system must update only the relevant knowledge (robot puts bread on plate - the wall colour should not change).
- **Frames** = structured info about objects/concepts (Car: make, model, colour, year).
- **Scripts** = typical sequences of events (Restaurant: ordering, eating, paying).
- **Frame-based system:** knowledge organised as templates; slots are populated when a new instance is met.

Challenges and Solutions

Challenge	Solution
Scalability issues	Careful planning, well-structured hierarchies, automated management/consistency tools
Complex relationships	Property chains / link slots; ontologies or graph databases
Inheritance overhead	Selective inheritance or override at specific levels
Expressiveness limitations	Hybrid systems (ontologies, semantic networks)
Inefficient retrieval	Indexing and caching; optimised retrieval algorithms
Lack of standardisation	Standard frame languages, best practices, standard ontologies/taxonomies

Frame Inheritance

Traits, features and connections pass from **parent frames to child frames**, which cuts repetition and improves decision-making. General attributes stay in the parent; specific ones are defined in the child. Updating a parent updates all its children. Slots can hold: explicit values, default values, procedures, references to other frames.

Type	Description	Example
Single inheritance	Child inherits from only one parent (linear)	Vehicle (engine type, speed, capacity) → Car (adds fuel type, doors)
Multiple inheritance	Child inherits from several parents	Land Vehicle + Electric Vehicle → Electric Car
Overriding inheritance	Child modifies/replaces inherited attributes	Vehicle speed 100 km/h, 5 passengers → Truck 80 km/h, 3 tons

Application	Frame inheritance example
Expert systems	Disease frame (symptoms, causes, treatments) → Influenza, Diabetes
NLP	Verb frame (tense, subject, object) → run, eat, write
Robotics	Vehicle frame → Self-Driving Car (adds sensor processing, ML decisions)
ML / data organisation	User Profile frame → Frequent Buyers, Casual Browsers

5. Quick Revision Sheet

Topic	Remember
PL vs FOL	PL = true/false only. FOL = predicates, variables, constants, quantifiers (∀, ∃)
Semantic net vs frame	Semantic net = graph of nodes and labelled links. Frame = template of slots and fillers
Forward vs backward chaining	Forward: facts → conclusion. Backward: goal → supporting facts
Production system	Rule base + working memory + inference engine (+ conflict resolution)
Modus Ponens / Tollens	$P \to Q, P \vdash Q$ / $P \to Q, \neg Q \vdash \neg P$
De Morgan	$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$; $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$
KR requirements	Representational adequacy, Inferential adequacy, Inferential efficiency, Acquisitional efficiency, Consistency
Frame inheritance types	Single, Multiple, Overriding
Conflict resolution	Specificity, Recency, Priority levels

Note: the last slide ("Practice Task") is an image and was not included in the extracted text - check it in the original PPT.